

Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !' DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !' MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !' FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !' QA-VALIDACION (GLX-QA-VALIDACION-V1)

1. ANÁLISIS DE ARQUITECTURA

Micro-ADR: Página Web Productora 2 (SCRUM-20)

1. CONTEXTO

Se requiere una presencia web para una productora audiovisual/creativa. El stack está explícitamente restringido a HTML5 + CSS + JS puro (sin frameworks de backend, sin Node.js, sin bases de datos). El formulario de contacto debe enviar correos sin almacenar datos. El equipo de diseño aún no entrega assets finales. El sitio es un MVP de presencia digital con 4 secciones: Home, Servicios, Quiénes Somos y Contacto.

2. DECISIÓN ARQUITECTÓNICA

Arquitectura elegida: Sitio Estático Multi-Página (Static Multi-Page Website)

- 4 archivos HTML independientes (una por sección/página), un archivo CSS global + módulos CSS por sección, y un archivo JS principal con módulos funcionales.
- El formulario de contacto utilizará un servicio de terceros tipo **Formspree.io** (integración vía ``action`` del form o fetch a su API) para el envío de correos sin backend propio. Esto cumple el requisito de no almacenar datos en base de datos y no requiere servidor.
- Las imágenes usarán exclusivamente URLs de ``https://placeholder.co`` y ``https://picsum.photos`` hasta la entrega de assets finales.

3. JUSTIFICACIÓN BASADA EN YAGNI / KISS / BORING TECHNOLOGY

- ****¿Por qué NO SPA (React/Vue/Angular)?**** El requerimiento prohíbe frameworks JS. Además, una SPA agrega complejidad de bundling, routing client-side y SEO penalties sin ningún beneficio para 4 páginas estáticas. YAGNI: no se necesita estado reactivo complejo.
- ****¿Por qué NO backend propio (Node/PHP)?**** El stack está restringido a HTML/CSS/JS. El envío de correos se delega a Formspree (servicio maduro, gratuito en tier básico, sin infraestructura propia). Esto elimina 100% de la superficie de ataque de inyección SQL/XSS en servidor.
- ****¿Por qué NO CMS headless (WordPress, Strapi)?**** El requerimiento menciona integración CMS como deseable pero el stack prohíbe backend. Se documenta como deuda técnica futura. Para el MVP, el contenido es estático en HTML.
- ****¿Por qué NO microservicios ni arquitectura distribuida?*** Es un sitio de 4 páginas. Aplicar microservicios aquí sería ingeniería astronáutica pura. KISS absoluto.
- ****¿Por qué Multi-Page y no Single-Page con anchors?*** Mejor SEO nativo, URLs limpias por sección, carga progresiva real (solo carga lo necesario), y separación de concerns más clara para mantenimiento futuro.

4. RIESGOS ASUMIDOS POR SIMPLIFICAR

- ****Riesgo 1 (Formspree):**** Dependencia de servicio externo para el formulario. Mitigación: Formspree tiene 99.9% uptime y el tier gratuito soporta 50 envíos/mes. Si el volumen crece, se migra a EmailJS o backend propio.
- ****Riesgo 2 (Sin CMS):**** El contenido es estático en HTML. Actualizar servicios requiere editar HTML directamente. Mitigación: Documentado como deuda técnica. La estructura modular del HTML facilita la futura integración con un CMS headless.
- ****Riesgo 3 (Multi-idioma):**** El requerimiento pide ES/EN. Se implementa con un switcher JS que intercambia atributos ``data-lang`` en el DOM, sin i18n library. Suficiente para MVP. Riesgo: mantenimiento manual de traducciones en JS object.
- ****Riesgo 4 (CDN):**** No se configura CDN en el código (es infraestructura de hosting). Se documenta como recomendación de deployment en README.
- ****Riesgo 5 (Seguridad XSS):**** Sin backend, el XSS se mitiga con sanitización JS en el cliente antes del submit y

Content-Security-Policy headers (configurables en el hosting). Se implementa validación estricta en JS.

5. CONTRATOS E INTERFACES

- ****Formspree endpoint:**** Configurar `FORMSPREE\_ENDPOINT` como constante en `js/config.js` (ej: `https://formspree.io/f/YOUR\_FORM\_ID`). El desarrollador debe reemplazar `YOUR\_FORM\_ID` con el ID real del proyecto en Formspree.
- ****Placeholder images:**** Todas las `` usan `https://placeholder.co/{W}x{H}` o `https://picsum.photos/{W}/{H}` como `src`. Nunca rutas locales.
- ****Idiomas:**** Objeto de traducciones en `js/i18n.js` con claves `es` y `en`.
- ****CSS Custom Properties:**** Variables de diseño centralizadas en `:root` dentro de `css/variables.css` (colores de marca, tipografías, espaciados).

2. MÉTRICAS DE ITERACIÓN (QA)

- productora-web/index.html | Estado: WARNING (Forzado) | Iteraciones: 3
- productora-web/servicios.html | Estado: APROBADO | Iteraciones: 1
- productora-web/quienes-somos.html | Estado: APROBADO | Iteraciones: 1
- productora-web/contacto.html | Estado: APROBADO | Iteraciones: 1
- productora-web/css/variables.css | Estado: APROBADO | Iteraciones: 1
- productora-web/css/global.css | Estado: APROBADO | Iteraciones: 1
- productora-web/css/sections.css | Estado: APROBADO | Iteraciones: 1
- productora-web/js/config.js | Estado: APROBADO | Iteraciones: 1
- productora-web/js/i18n.js | Estado: APROBADO | Iteraciones: 1
- productora-web/js/form.js | Estado: APROBADO | Iteraciones: 1
- productora-web/js/main.js | Estado: APROBADO | Iteraciones: 1
- productora-web/README.md | Estado: APROBADO | Iteraciones: 1

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 169,213

Tokens de salida (output): 57,066

Total tokens: 226,279

Horas-hombre estimadas: ~19.8 horas

Modelo utilizado: claude-sonnet-4-6

Desglose por Agente:

DISEÑO-ARQUITECTURA: !"4,167 input | !"1,836 output | 1 llamadas

MICROPLANIFICACION-DISEÑO: !"30,659 input | !"17,897 output | 12 llamadas

FRONTEND-DESARROLLO: !"51,767 input | !"30,867 output | 14 llamadas

QA-VALIDACION: !"82,620 input | !"6,466 output | 14 llamadas